SOFTWARE ENGINEERING 1 · MID-SEMESTER PRESENTATION

Tutti

A peer-to-peer tutoring platform

Deliverable 1 — Requirements and modelling

Nicolle · Anastasiia · Nikita · Stefan · Yanislav

THE IDEA

Tutti in one slide

Students who struggle find an approved peer tutor, request a session and get the answer by e-mail. Coordinators keep the quality under control.

FIVE DECISIONS THAT SHAPE THE MODEL

Free

No payments: tutoring is a peer service

One-to-one

One tutor and one student per session

Request and accept

The tutor accepts or declines; requests expire after 48 h

E-mail only

Every update reaches users by e-mail

Per faculty

One coordinator approves tutors with at least 15/20

AGENDA

Who presents what

1	The idea, stakeholders and requirements	Nicolle
2	User stories and story maps	Anastasiia
3	How we work and the use-case model	Nikita
4	The domain model	Stefan
5	The booking flow in sequence	Yanislav

REQUIREMENTS ELICITATION

Seven stakeholders, four categories

PRIMARY

Student
Find an approved tutor at a time that fits

Tutor
Set own hours; no late cancellations

Coordinator
Only qualified tutors, fast action on issues

SECONDARY

Lecturers
Extra help for students, but no graded work done for them

Sponsor
Visibility and anonymous usage reports

EXTERNAL

Data Protection Officer
Grades only for the coordinator; minimal data

HIDDEN

Development team
Clear priorities; modular, testable code

REQUIREMENTS ANALYSIS

Requirements at a glance

27 functional

12 non-functional



19 Must · 3 Should · 2 Could · 3 Won't

A CONFLICT WE RESOLVED

Students want to cancel late; tutors want notice. Cancelling is always possible, but under 24 h it counts as late, and 2 late cancellations or no-shows per semester flag the user.

PRIVACY · GDPR

Grades are visible only to the coordinator

USABILITY

A session request in at most 3 steps

PERFORMANCE

Results within 2 s for 200 concurrent users

USER STORIES

Fifteen stories, one goal each

As a **role**, I want **a goal** so that **a reason**

S2 · Student

As a student, I want to request a session by choosing a free time, a duration, online or onsite, a purpose and a description so that the tutor knows what I need before accepting.

T3 · Tutor

As a tutor, I want to accept or decline session requests, giving a reason when I decline, so that I only commit to sessions I can give.

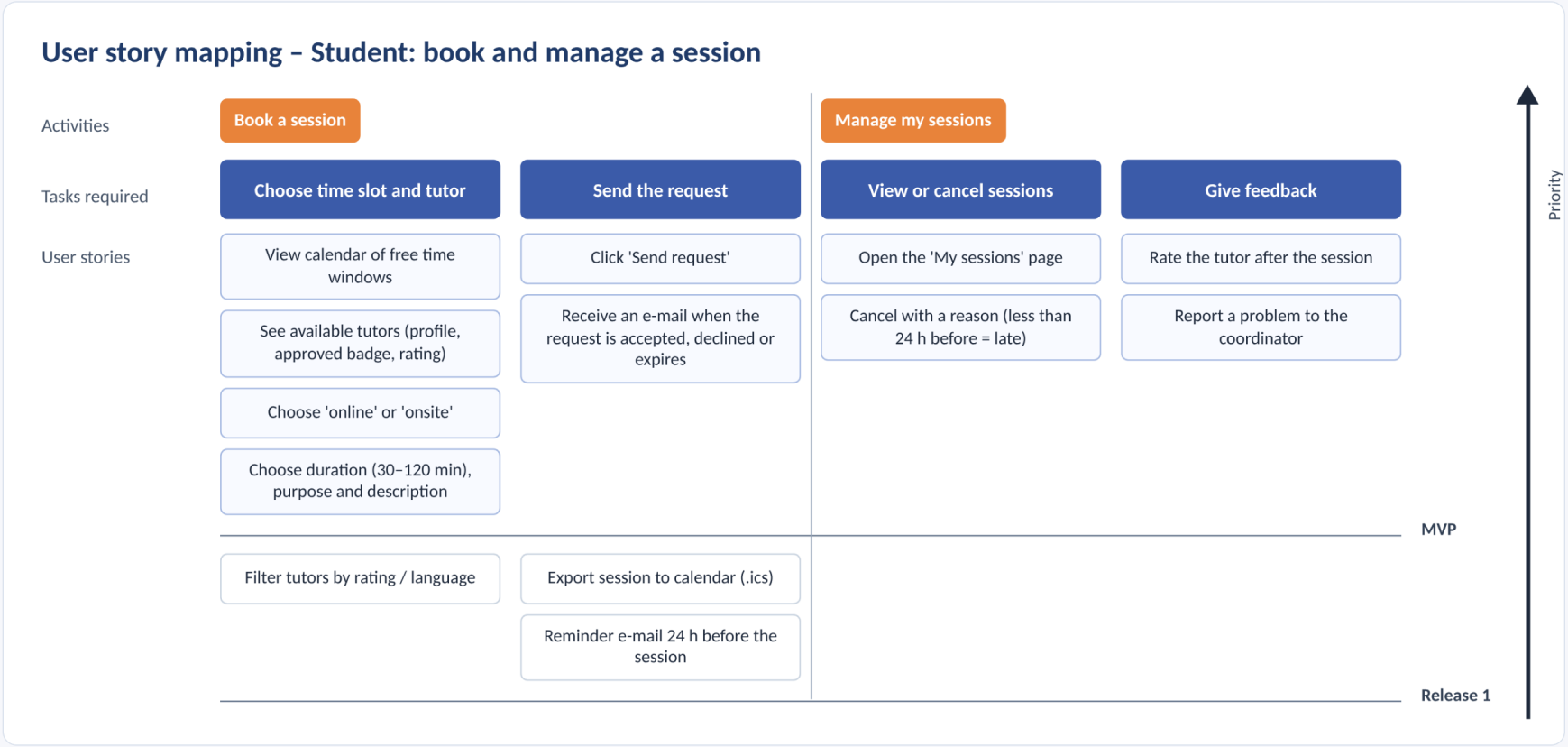
C1 · Coordinator

As a coordinator, I want to approve or reject tutor applications from my faculty so that only students with at least 15/20 in a subject tutor it.

5 stories per role, labelled S1–S5, T1–T5 and C1–C5, and traced to the functional requirements.

USER STORY MAPPING

Story map: book and manage a session



Left to right

The journey: activities, then tasks, then stories

Top to bottom

Priority: above the line is the MVP, below it Release 1

Five maps

One per journey, for students, tutors and coordinators

PROJECT PLANNING

How we work: lightweight Agile

Kanban-style flow

To do → in progress → review → done, inside each deliverable's time-box

User stories as backlog

Prioritised with MoSCoW; the story maps draw the MVP line

Responding to change

Payments dropped, booking became request and accept, every message by e-mail

Clear ownership

One artifact per person, synced and decided in a regular team meeting

Tools

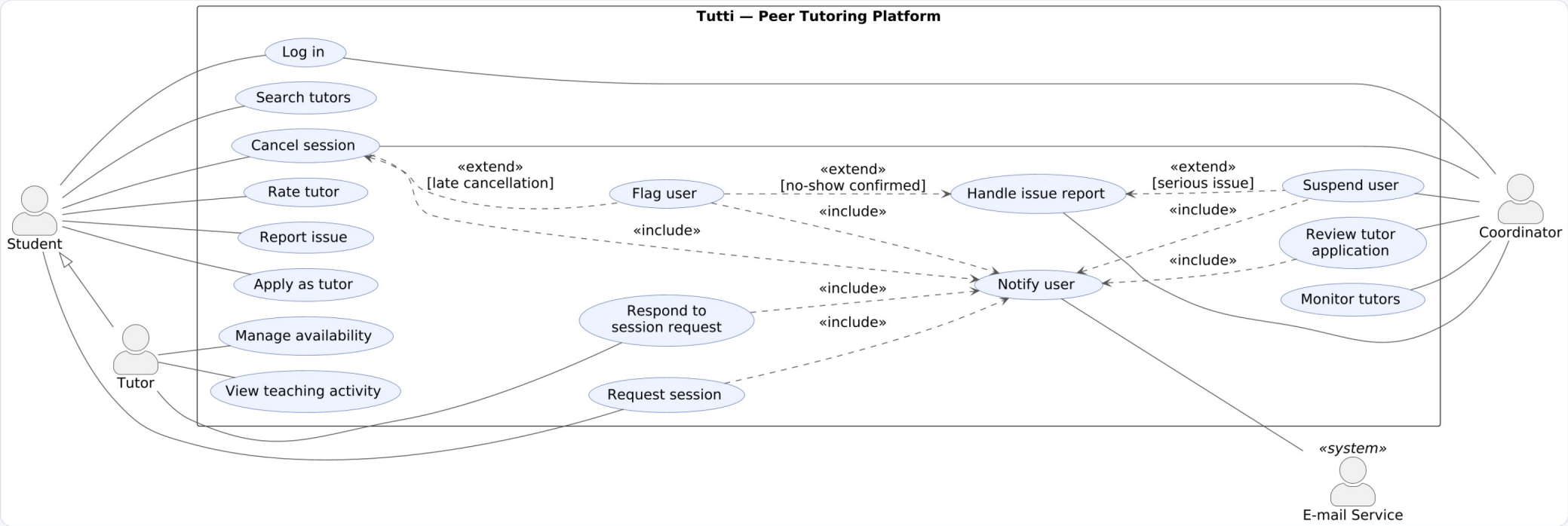
GitHub repo with the deliverable and every diagram as PlantUML text. AI for drafts: the team reviewed, changed and verified each result.

Retrospective

Went well: early decisions, models cross-checked. To improve: the story maps drifted after decisions, so "done" now includes a consistency check.

USE-CASE MODEL · 16 USE CASES

Use-case model



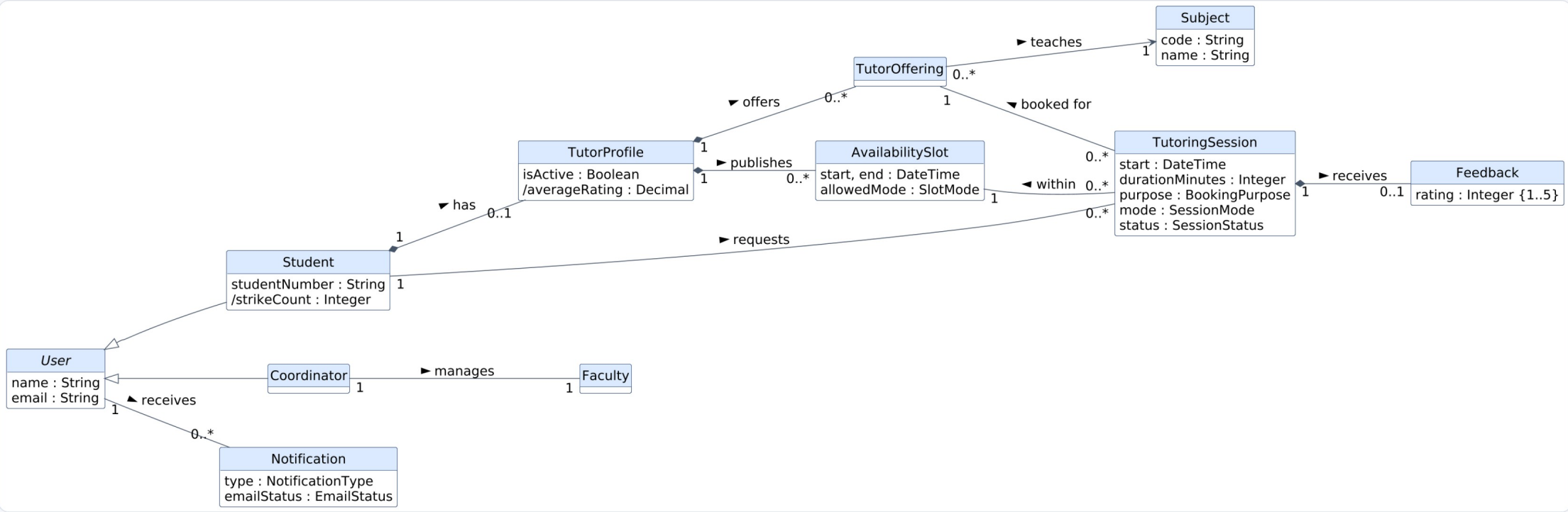
Tutor specialises Student: a tutor can also book sessions

«include» Notify user: six use cases always send an e-mail

«extend» Flag, Suspend: only when a rule is broken

DOMAIN MODEL

Domain model: the core



Core excerpt: 11 of the 18 classes. The full model, with 9 enumerations and all constraints, is Figure 7 in Deliverable 1.

DOMAIN MODEL

Four design decisions

01

A tutor is a role

A student has 0..1 tutor profile (composition), so the same person can learn and teach.

02

Rules live in the model

Introduction: 30 min; other purposes: 60, 90 or 120 min. Sessions in one time window never overlap.

03

Derived, not stored

/averageRating, /hoursTaught and /strikeCount are computed from sessions and feedback.

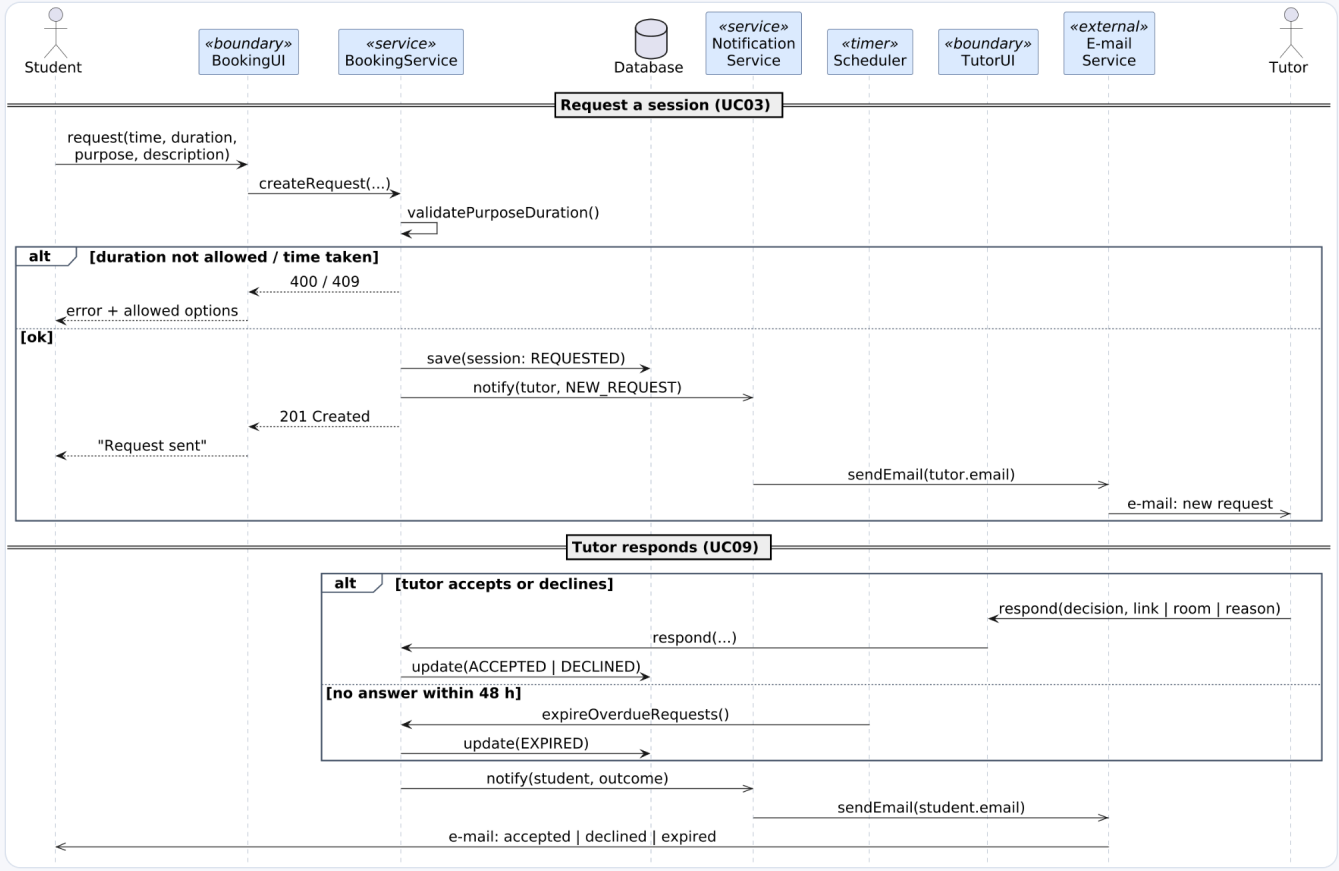
04

Every e-mail is recorded

A notification keeps its type and its e-mail status: pending, sent or failed.

BEHAVIOURAL MODEL

The booking flow in sequence



Layers

Boundary, service, database: the UI never touches the data directly

alt fragments

Errors (400 and 409) and the three outcomes of a request

Asynchronous e-mails

Open arrowheads: nobody waits for an e-mail

Simplified view

The full diagram has 84 numbered messages (Deliverable 1, Figures 8–10)

BEHAVIOURAL MODEL

Three ways a request ends

ACCEPTED

The tutor accepts

Adds a meeting link or a room

E-mail to the student

Date, time, duration, mode, link or room

DECLINED

The tutor declines

Gives a reason; the time is freed

E-mail to the student

The reason and an invitation to request another time

EXPIRED

Nobody answers

After 48 h, or 24 h before the start, the Scheduler frees the time

E-mail to the student

An invitation to request again

Answering a request that already expired or was cancelled returns 409 “This request is no longer pending”: nothing changes and no e-mail is sent.

WRAP-UP

One consistent model



Every decision, from free and one-to-one to request-and-accept and e-mail only, is traced through all six.

Questions?

Deliverable 1 and every diagram source are in our GitHub repository